BÁO CÁO TIẾN ĐỘ THỰC HIỆN ĐỒ ÁN NGÀNH

Nguyễn Việt Linh - Vũ Hàn Tín

Tuần 6 – Tuần 7 (Tháng 10/2026)

1 Báo cáo tiến độ tuần 6–7

1.1 Mục tiêu thực hiện

Trong tuần 6 và tuần 7, nội dung thực hiện tập trung vào việc tìm hiểu cơ sở lý thuyết về mạng nơ-ron tích chập (Convolutional Neural Network – CNN), phép tích chập theo chiều sâu (Depthwise Convolution) và các kiến trúc phần cứng có khả năng triển khai phép toán này trên FPGA.

Mục tiêu của giai đoạn này bao gồm:

- Tìm hiểu nguyên lý hoạt động của phép tích chập trong CNN và vai trò của các kernel trong quá trình trích xuất đặc trưng.
- Phân tích sự khác biệt giữa convolution thông thường, depthwise convolution và depthwise separable convolution.
- Khảo sát các thành phần phần cứng phục vụ tính toán convolution, bao gồm bộ nhân–cộng (MAC), bộ đệm dòng (line buffer), cửa sổ trượt (sliding window) và cơ chế xử lý dữ liệu theo đường ống (pipeline).
- Xác định bài toán, phạm vi triển khai và phương pháp kiểm chứng thiết kế phần cứng bằng mô hình tham chiếu Python.

1.2 Cơ sở lý thuyết

1.2.1 Mạng nơ-ron tích chập

Mạng nơ-ron tích chập là một kiến trúc mạng nơ-ron sâu được sử dụng rộng rãi trong các bài toán xử lý ảnh. Đầu vào của một lớp convolution thường được biểu diễn dưới dạng tensor ba chiều:

$$X \in \mathbb{R}^{H \times W \times C_{\text{in}}}, \tag{1}$$

trong đó H, W lần lượt là chiều cao, chiều rộng của feature map và C_{in} là số kênh đầu vào.

Phép convolution sử dụng các kernel có trọng số xác định để thực hiện phép nhân và cộng trên các vùng cục bộ của đầu vào. Kết quả thu được là các feature map biểu diễn mức độ phản hồi của kernel tại từng vị trí không gian.

Đối với convolution thông thường, mỗi kernel kết hợp thông tin từ tất cả các kênh đầu vào để tạo ra một kênh đầu ra. Đây là cơ sở để phân biệt convolution thông thường với depthwise convolution.

1.2.2 Depthwise Convolution

Depthwise convolution thực hiện phép tích chập độc lập trên từng kênh đầu vào. Với depth multiplier bằng 1, mỗi kênh đầu vào tương ứng với một kernel không gian và tạo ra một kênh đầu ra.

Đầu vào được biểu diễn bởi:

$$X \in \mathbb{R}^{H \times W \times C}, \tag{2}$$

và bộ kernel bởi:

$$K \in \mathbb{R}^{K_h \times K_w \times C}. \tag{3}$$

Phép tính tại vị trí (i, j) trên kênh c được xác định bởi:

$$Y(i, j, c) = \sum_{u=0}^{K_h-1} \sum_{v=0}^{K_w-1} X(i+u, j+v, c) K(u, v, c). \tag{4}$$

Công thức trên cho thấy phép tính chỉ sử dụng dữ liệu thuộc cùng một kênh, không thực hiện cộng gộp trực tiếp giữa các kênh đầu vào.

Với stride bằng 1 và không sử dụng padding, kích thước đầu ra là:

$$H_{\text{out}} = H - K_h + 1, \qquad W_{\text{out}} = W - K_w + 1. \tag{5}$$

Số phép nhân–cộng (MAC) cần thiết cho toàn bộ đầu ra trong trường hợp này là:

$$N_{\text{MAC}} = H_{\text{out}} W_{\text{out}} C K_h K_w. \tag{6}$$

1.2.3 So sánh với convolution thông thường

Với cùng kích thước đầu vào, kernel và số kênh đầu ra C_{out}, convolution thông thường cần:

$$N_{\text{MAC,std}} = H_{\text{out}} W_{\text{out}} K_h K_w C_{\text{in}} C_{\text{out}}. \tag{7}$$

Trong khi đó, depthwise convolution với depth multiplier bằng 1 cần:

$$N_{\text{MAC,dw}} = H_{\text{out}} W_{\text{out}} K_h K_w C_{\text{in}}. \tag{8}$$

Do đó, depthwise convolution có số phép tính thấp hơn đáng kể trong các trường hợp số kênh đầu ra của convolution thông thường lớn. Tuy nhiên, hai phép toán không hoàn toàn tương đương về chức năng: depthwise convolution không trực tiếp kết hợp thông tin giữa các kênh.

Depthwise separable convolution bổ sung một lớp pointwise convolution với kernel 1×1 sau depthwise convolution. Bước pointwise thực hiện kết hợp thông tin giữa các kênh và cho phép tạo ra số kênh đầu ra theo yêu cầu. Trong phạm vi báo cáo này, nội dung triển khai phần cứng được giới hạn ở phép depthwise convolution.

1.3 Khảo sát kiến trúc phần cứng liên quan

Để triển khai depthwise convolution trên FPGA, một số thành phần kiến trúc phần cứng được xem xét như sau:

- **Bộ nhân–cộng (MAC):** Thực hiện phép nhân giữa pixel và trọng số kernel, sau đó cộng dồn các tích để tạo ra một giá trị đầu ra.
- **Bộ đệm dòng (line buffer):** Lưu trữ một số dòng dữ liệu đầu vào nhằm tái sử dụng các pixel khi tạo cửa sổ convolution, qua đó giảm nhu cầu đọc lại dữ liệu từ bộ nhớ ngoài.
- **Cửa sổ trượt (sliding window):** Tổ chức các pixel lân cận thành một cửa sổ có kích thước bằng kernel. Cửa sổ được cập nhật khi dữ liệu đầu vào được đưa vào mạch.
- **Đường ống xử lý (pipeline):** Chia quá trình tính toán thành các giai đoạn liên tiếp để cải thiện khả năng xử lý dữ liệu theo luồng.
- **Bộ điều khiển:** Quản lý quá trình nhận dữ liệu, cập nhật cửa sổ, tích lũy kết quả và phát tín hiệu đầu ra hợp lệ.

Các tài liệu về accelerator CNN trên FPGA cho thấy việc tái sử dụng dữ liệu, tổ chức bộ đệm và khai thác song song là những yếu tố quan trọng bên cạnh số lượng phép toán số học. Vì vậy, việc thiết kế cần cân bằng giữa tài nguyên logic, bộ nhớ nội và hiệu năng xử lý.

1.4 Xác định bài toán và phạm vi thiết kế

Từ cơ sở lý thuyết đã tìm hiểu, bài toán được xác định là xây dựng một module phần cứng thực hiện phép depthwise convolution trên FPGA và kiểm chứng tính đúng đắn của kết quả bằng mô hình tham chiếu phần mềm.

Phạm vi thiết kế ban đầu được giới hạn như sau:

- Đầu vào là tensor có kích thước $H \times W \times C$.
- Kernel có kích thước 3×3, mỗi kênh đầu vào sử dụng một kernel riêng.
- Depth multiplier bằng 1, stride bằng 1 và padding bằng 0.
- Không xét bias, hàm kích hoạt hoặc pointwise convolution trong phiên bản đầu tiên.
- Phép tính số học được triển khai theo biểu diễn số có dấu với độ rộng bit xác định; độ rộng bộ tích lũy phải đủ để hạn chế hiện tượng tràn số.

Với các giả thiết trên, đầu ra của mỗi kênh có kích thước $(H-2) \times (W-2)$. Module cần thực hiện phép nhân giữa từng pixel trong cửa sổ và trọng số tương ứng, sau đó cộng chín tích để tạo ra một phần tử đầu ra.

1.5 Phương pháp triển khai và kiểm chứng

Phương pháp kiểm chứng dự kiến sử dụng mô hình tham chiếu phần mềm (Python golden reference model) kết hợp với mô phỏng RTL bằng SystemVerilog.

Quy trình thực hiện gồm các bước:

1. Xây dựng chương trình Python để sinh tensor đầu vào và kernel thử nghiệm, đồng thời tính kết quả depthwise convolution theo công thức toán học.
2. Xây dựng module RTL bằng SystemVerilog để thực hiện phép tính với cùng dữ liệu đầu vào và cùng quy ước số học.
3. Xây dựng testbench để cung cấp dữ liệu cho module, thu thập kết quả đầu ra và so sánh với kết quả tham chiếu từ Python.
4. Kiểm tra nhiều trường hợp dữ liệu, bao gồm dữ liệu ngẫu nhiên, dữ liệu toàn số 0, giá trị âm và các trường hợp biên.
5. Sau khi kết quả mô phỏng đạt yêu cầu, tiến hành tổng hợp và triển khai trên FPGA để đánh giá tài nguyên và hiệu năng.

Để bảo đảm phép so sánh có ý nghĩa, mô hình Python và RTL cần thống nhất về thứ tự các chiều tensor, cách đánh chỉ số, quy tắc padding, độ rộng bit, xử lý số có dấu và thời điểm phát kết quả đầu ra.

1.6 Yêu cầu thiết kế

Các yêu cầu chính đối với thiết kế được xác định như sau:

- **Yêu cầu chức năng:** Thực hiện đúng phép depthwise convolution theo cấu hình đã xác định.
- **Yêu cầu số học:** Bảo đảm độ rộng dữ liệu và bộ tích lũy phù hợp; xử lý nhất quán các giá trị âm và hiện tượng tràn số.
- **Yêu cầu điều khiển:** Quản lý đúng quá trình nhận dữ liệu, cập nhật cửa sổ trượt và xác định thời điểm đầu ra hợp lệ.
- **Yêu cầu kiểm chứng:** Kết quả RTL phải khớp với mô hình tham chiếu Python trong các trường hợp kiểm thử đã xác định.
- **Yêu cầu triển khai:** Thiết kế có thể tổng hợp lên FPGA và cho phép thống kê tài nguyên sử dụng, độ trễ xử lý và thông lượng sau khi hoàn thành triển khai.

1.7 Kết quả và kế hoạch tiếp theo

Trong giai đoạn tuần 6–7, trọng tâm công việc là tìm hiểu cơ sở lý thuyết, phân tích phép depthwise convolution, khảo sát các thành phần phần cứng liên quan và xác định phạm vi thiết kế cũng như phương pháp kiểm chứng.

Ở giai đoạn tiếp theo, công việc dự kiến bao gồm xây dựng mô hình tham chiếu Python, thiết kế module depthwise convolution bằng SystemVerilog, phát triển testbench và kiểm tra kết quả mô phỏng. Sau khi hoàn thành kiểm chứng chức năng, thiết kế sẽ được tổng hợp và đánh giá trên FPGA theo các chỉ tiêu tài nguyên và hiệu năng.